

Business Requirements Document (BRD)

Project: Orders and Customer Data Pipeline — Delta Live Tables Implementation

Prepared by: Senior Business Analyst

Date: 2026-03-30

Version: 1.0

1. Executive Summary

Purpose

- Define business and system-level requirements to extract, transform, load and maintain customer and order data into Delta Lake using Delta Live Tables (DLT).
- Provide a reliable, auditable, and history-preserving data product: an ordersummary table (SCD Type 2) and a customeraggregatespend table (daily aggregated spend).

Scope

- Ingest CSV source files for customer and order data into Delta tables.
- Cleanse data (remove NULLs and duplicates).
- Compute order TotalAmount.
- Create and maintain an SCD Type 2 ordersummary table (preserving customer history).
- Produce a per-customer per-day aggregated spend table named customeraggregatespend.
- Implement the pipeline using Delta Live Tables on the gen_ai_poc_databrickscoe catalog and sdlc_wizard schema.

Out of Scope

- Source system changes upstream of provided CSV.
- BI dashboarding, consumption layer SLAs, or downstream application development (beyond the aggregate table).
- Real-time streaming ingestion beyond DLT incremental processing (unless specified as part of DLT configuration).

2. Background and Business Context

- Business stakeholders need accurate historical customer/order relationships and daily aggregated spend per customer for analytics, reporting and trend analysis.
- Historical changes to customer attributes (e.g., Name, EmailId, Region) must be tracked to enable point-in-time analysis and correct attribution of orders.

3. Objectives and Success Criteria

Objectives

- Load source CSV data into Delta tables for customer and order.
- Enrich order data with $\text{TotalAmount} = \text{PricePerUnit} * \text{Qty}$.
- Cleanse data (remove NULL/"Null" values and duplicates).
- Create and maintain ordersummary as SCD Type 2 capturing customer attribute changes.
- Produce customeraggregatespend summarizing TotalAmount by customer Name and Date.
- Implement solution using Delta Live Tables to enforce pipeline reliability, incremental refreshes and monitoring.

Success Criteria

- Orders and customer Delta tables created and populated with known sample data.
- TotalAmount calculated correctly for every order row.
- No rows with NULL or string "Null" in critical fields remain; duplicates eliminated.
- ordersummary implements SCD Type 2: active indicator, StartDate, EndDate maintained; historical records retained when customer attributes change.
- Aggregates in customeraggregatespend reflect sum of TotalAmount grouped by Name and Date and are refreshed incrementally.
- End-to-end pipeline implemented on Delta Live Tables with documented DLT pipeline configuration and tested runs.

4. Stakeholders and Roles

- Business Sponsor: Owner of reporting/analytics requirements
- Product Owner/Analyst: Approves requirements, acceptance criteria
- Data Engineer (DLT): Implementation, deployment and operationalization on Databricks
- Data Architect: Schema design, catalog/schema governance (gen_ai_poc_databrickscoe.sdmc_wizard)
- QA/Tester: Data quality testing, acceptance testing
- Data Governance/Compliance: Approve retention, PII handling, and access
- Operations/Monitoring: Maintain DLT pipeline, SLAs and alerts

5. Source Data (Input)

Source files and initial ingestion

- Source format: CSV
- Initial ingestion path(s) (as provided):
- customer data: /Volumes/gen_ai_poc_databrickscoe/sdmc_wizard/customerdata
- order data: /Volumes/gen_ai_poc_databrickscoe/sdmc_wizard/orderdata
- Source schemas (as provided):
- customer:

- CustId (primary key)
- Name
- EmailId
- Region
- order:
- OrderId (primary key)
- ItemName
- PricePerUnit
- Qty
- Date (order date)
- CustId (foreign key to customer.CustId)

Notes/Assumptions

- Date field format must be validated and standardized (e.g., ISO 8601 yyyy-MM-dd). If time component present, define handling.
- PricePerUnit and Qty must be numeric. If they are strings (or contain commas/currency symbols), transformation rules must be defined.
- CustId is considered unique in customer CSV and used to join orders to customers.

6. Data Quality and Cleansing Requirements

- Remove rows that contain NULL values or literal string "Null" in critical columns:
- customer: drop records where CustId is NULL/"Null" or where required attributes (Name, EmailId, Region) are NULL/"Null" as per data quality policy.
- order: drop records where OrderId, CustId, PricePerUnit or Qty are NULL/"Null", or Date is NULL/"Null".
- Remove duplicate records in both customer and order Delta tables. Definition of duplicate:
- customer: duplicate if CustId duplicates with all attributes identical OR if a deduplication policy exists (e.g., keep latest by ingest timestamp).
- order: duplicate if OrderId duplicates, keep single authoritative record (prefer latest ingest or non-null fields).
- Logging: Track number of rows dropped due to NULL/"Null" and duplicates; persist audit metrics in pipeline monitoring logs.
- Data type enforcement: convert PricePerUnit to decimal(precision) and Qty to integer, with documented rounding/truncation rules. Validate out-of-range or negative values and handle per policy (reject or flag).

7. Transformations and Business Logic

7.1 Compute TotalAmount

- New calculated column on order table:

- name: TotalAmount
- expression: PricePerUnit * Qty
- Type: decimal/float as appropriate. Recommended decimal(18,2).
- Null handling: If either PricePerUnit or Qty is null after cleansing, the record should be excluded per cleansing rules.

7.2 Join and Target: ordersummary (SCD Type 2)

- Target catalog/schema/table:
- Catalog: gen_ai_poc_databrickscoe
- Schema: sdlc_wizard
- Table: ordersummary
- ordersummary logical schema (initial):
- CustId
- Name
- EmailId
- Region
- OrderId
- ItemName
- PricePerUnit
- Qty
- Date
- Enrichment: Include TotalAmount in ordersummary (derived) — business requirement implies TotalAmount calculation exists on order and should be available in ordersummary and used in aggregates.
- Add SCD control columns required for Type 2:
- IsActive (boolean) — current active record indicator
- StartDate (timestamp/date) — effective start of the record version
- EndDate (timestamp/date) — effective end of the record version (null for active)
- RecordHash or BusinessKeyHash (optional) — hash of tracked attributes to quickly detect changes
- EffectiveTimestamp or IngestTimestamp — to maintain ordering on changes
- Primary/business key:
- Natural keys: CustId + OrderId (to uniquely identify an order for a customer)
- Surrogate key: optionally add OrderSummaryId for uniqueness of SCD rows.
- Join condition: customer.CustId = order.CustId

7.3 SCD Type 2 Behavior (Rules)

- Detect customer attribute changes (tracked attributes: Name, EmailId, Region). Changes are detected on the key CustId.
- When a change is detected for a given CustId:
- Mark the existing active record(s) for that CustId in ordersummary IsActive = false and set EndDate to the change effective date (StartDate previously set).
- Insert a new record for each existing order row representing the new customer attributes with IsActive = true and StartDate = change effective date, EndDate = NULL.
- If multiple orders exist for CustId, SCD update must ensure that each order row gets the appropriate historical version to preserve the relationship between order and the customer attributes at the time of order or change depending on business rules:
- Standard Type 2 semantics: when customer attributes change, new versions should reflect the current attributes going forward; historical orders retain the version applicable.
- Important decision (requires PO confirmation): whether SCD changes are applied retroactively to existing orders (i.e., change affects only new orders) or whether the SCD versioning applies as of change time across all orders. Clarify desired behavior:
- Option A (common): SCD versioning captures customer attribute validity over time; orders are associated with the customer version valid at the order Date.
- Option B: simpler: each order row stores a snapshot of customer attributes at ingestion time; SCD Type 2 still tracks customer records independently of orders.
- StartDate/EndDate population:
- On insert: StartDate = current processing timestamp or event effective date; EndDate = NULL; IsActive = true.
- On deactivate: EndDate = processing timestamp or event effective date; IsActive = false.
- Reconciliation and deduplication of change events:
- Use record hashing and timestamp compare to avoid unnecessary churn when no real attribute change occurs.

7.4 Aggregation: customeraggregatespend

- Target:
- Catalog: gen_ai_poc_databrickscoe
- Schema: sdlc_wizard
- Table: customeraggregatespend
- Schema:
- Name
- TotalAmount (sum of TotalAmount)
- Date
- Business rule:
- Aggregate TotalAmount from ordersummary grouped by Name and Date.

- Use ordersummary records applicable for a given Date. Clarify whether aggregation should consider the Name from active SCD records as of the order Date. (Recommend: use the customer attribute version valid at the order Date.)
- Aggregation frequency:
 - Daily aggregation, refreshed incrementally each DLT run (or scheduled cadence).
- Data freshness:
 - Aggregates must reflect new orders and SCD changes as they are processed by the pipeline.
- Conflict resolution:
 - If Name changes across versions and orders are aggregated by Name, ensure orders are grouped by the Name value that was current at the order Date to avoid misattribution.
- Partitioning:
 - Partition by Date for performance.

8. Data Model and Schema Definitions

8.1 Raw Delta Tables (Staging)

- customer_delta (staging; cleansed)
 - CustId (string/integer) — primary key
 - Name (string)
 - EmailId (string)
 - Region (string)
 - IngestTimestamp (timestamp)
 - SourceFile / SourceRowId (optional)
- order_delta (staging; cleansed)
 - OrderId (string/integer) — primary key
 - ItemName (string)
 - PricePerUnit (decimal(18,2))
 - Qty (integer)
 - Date (date)
 - CustId (string/integer)
 - TotalAmount (decimal(18,2))
 - IngestTimestamp (timestamp)

8.2 ordersummary (SCD Type 2)

- Suggested schema extensions:
 - OrderSummaryId (surrogate key) — optional
 - OrderId

- CustId
- Name
- EmailId
- Region
- ItemName
- PricePerUnit
- Qty
- TotalAmount
- Date
- IsActive (boolean)
- StartDate (timestamp/date)
- EndDate (timestamp/date)
- RecordHash (string) — hash of tracked attributes: Name|EmailId|Region
- IngestTimestamp (timestamp)
- Partitioning: by Date and IsActive or Date only (recommend partition by Date).
- Indexing/Optimization: Use Z-Ordering on CustId or OrderId for query performance in Databricks.

8.3 customeraggregatespend

- Schema:
- Name (string)
- Date (date)
- TotalAmount (decimal(18,2))
- Primary key: Name + Date
- Partitioning: by Date (recommended)
- Granularity: daily per Name

9. Process Flow / Functional Requirements (High-level steps)

- Step 1 — Ingest:
- Read source CSV files and load into Delta tables: customer_delta and order_delta at paths:
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata
- Register/ensure existence of Delta tables in catalog
gen_ai_poc_databrickscoe.sdlc_wizard.customer and .order.
- Step 2 — Schema enforcement:
- Apply schema mapping to source fields with type conversions and validations as defined.
- Step 3 — Compute TotalAmount:

- Add column TotalAmount = PricePerUnit * Qty to order_delta.
- Step 4 — Data cleansing:
 - Remove rows with NULL or literal string "Null" in critical fields.
 - Remove duplicates according to deduplication rules.
 - Capture audit counts of rejected/deduped rows.
- Step 5 — Ensure ordersummary table exists:
 - Create gen_ai_poc_databrickscoe.sdmc_wizard.ordersummary if not present with SCD columns.
- Step 6 — Join and load into SCD2:
 - Join customer and order data on CustId; determine change detection logic between incoming customer snapshot and current active customer version in ordersummary.
 - Implement SCD Type 2 semantics to update ordersummary (deactivate old versions and insert new ones when customer attributes change).
- Step 7 — ordersummary schema:
 - Confirm and deploy the ordersummary schema including SCD control columns.
- Step 8 — SCD handling:
 - Implement SCD Type 2 behavior as defined in section 7.3.
- Step 9 — Ensure customeraggregatespend exists:
 - Create gen_ai_poc_databrickscoe.sdmc_wizard.customeraggregatespend if not present.
- Step 10 — Aggregate and load:
 - Aggregate TotalAmount from ordersummary grouped by Name and Date and write to customeraggregatespend (incremental refresh).
- Step 11 — Delta Live Tables:
 - Implement steps 1–10 as a Delta Live Tables pipeline to provide declarative ETL/ELT, monitoring, and policy enforcement.

10. Non-Functional Requirements

Performance

- Pipeline should be able to handle daily volume expected (specify volumes: rows/day) — to be confirmed with stakeholders.
- Aggregation job should complete within defined SLA (e.g., 30 minutes after data availability).

Reliability & Availability

- DLT should be configured with retries and alerting; pipeline run success/failure logs must be persisted.

Scalability

- Solution must scale linearly with data growth; use partitioning and Delta optimizations (Z-Ordering/Compaction).

Security & Compliance

- Data stored in `gen_ai_poc_databricksco.sdlc_wizard` must follow access controls, encryption at rest and in transit.
- PII (EmailId) must be governed; masking/pseudonymization rules required if necessary for downstream exposures.

Auditability & Lineage

- Maintain audit logs for ingested rows, rows dropped, transformation counts and DLT run metadata.
- Enable Unity Catalog (if available) for lineage and access control.

Retention & Purging

- Define retention policy for historical SCD records if different from indefinite history. Document retention period (e.g., retain SCD history for 7 years).

Monitoring & Alerting

- DLT pipeline health must be monitored. Configure alerts for job failures, data volume anomalies, data quality rule violations (e.g., % nulls exceeding threshold).

11. Implementation Requirements and Constraints

- Implementation Framework: Delta Live Tables (DLT) on Databricks with target catalog `gen_ai_poc_databricksco` and schema `sdlc_wizard`.
- Table creation: ensure `CREATE TABLE IF NOT EXISTS` for `ordersummary` and `customeraggregatespend`.
- Idempotency: pipeline must be idempotent; handle reprocessing without creating duplicate SCD rows.
- Transactions: use Delta ACID transactions to ensure consistent updates.
- Concurrency: handle concurrent runs; use locks or mechanisms provided by DLT to prevent race conditions.
- Schema evolution: Support adding non-breaking columns; define policy for schema changes.
- Metadata: maintain lineage metadata in DLT and catalog.

12. Acceptance Criteria and Test Cases

Acceptance Criteria

- Source CSVs are successfully ingested to staging Delta tables with expected row counts.
- TotalAmount computed for each order equals $\text{PricePerUnit} * \text{Qty}$ with correct data types.
- All records with NULL/"Null" in critical fields removed; duplicates eliminated and reconciled.
- `ordersummary` table exists with SCD Type 2 behavior:

- When a customer's attributes change, previous active rows are marked inactive with EndDate set and new rows are inserted as active with StartDate set.
- Historical rows retained.
- customeraggregatespend contains aggregated TotalAmount by Name and Date consistent with ordersummary.
- DLT pipeline can be triggered end-to-end and completes successfully.

Representative Test Cases

- Ingest sample customers and orders; verify staging table contents, deduplication and null handling.
- Update a customer's Name and run pipeline; verify ordersummary inactive old records and new active records created with StartDate/EndDate set.
- Create orders for a customer before and after a Name change; aggregate by Name and Date and validate that aggregations attribute to the correct Name per business rule.
- Re-run pipeline on identical input and validate idempotency (no duplicate SCD rows).
- Introduce an invalid PricePerUnit (non-numeric), validate pipeline rejects or flags per policy.

13. Data Migration / Cutover Plan

- Initial load:
- Load historical CSVs into staging; run DLT pipeline in backfill mode to populate ordersummary and customeraggregatespend.
- Validation:
- Run reconciliation queries to validate row counts, amounts, and SCD versions vs. source snapshots.
- Cutover:
- After validation sign-off, enable scheduled DLT runs for ongoing incrementals.
- Rollback:
- Keep backups of Delta tables (time-travel enabled). Define rollback plan using Delta time travel if critical issues found.

14. Operational Considerations

- DLT Pipeline Configuration:
- Define cluster size, autoscaling, and job schedule.
- Define quality rules as DLT expectation constraints to fail or quarantine bad data.
- Logging & Metrics:
- Persist counts of ingested rows, dropped rows, duplicates, SCD changes, and aggregated output counts.
- Alerts:
- Configure alerts on DLT failures, data quality violations, or SLA misses.
- Maintenance:

- Periodic compaction (OPTIMIZE) and vacuum retention policy for Delta tables.
- Cost management:
- Estimate cluster costs and optimize frequency of runs and processing windows.

15. Security & Governance

- Access controls:
- Enforce principle of least privilege via Unity Catalog or Databricks ACLs for `gen_ai_poc_databrickscoe.sdlc_wizard`.
- PII handling:
- EmailId considered PII; define masking or restricted access for consumer groups if required.
- Data lineage:
- Use DLT built-in lineage and catalog metadata to maintain lineage for audits.

16. Risks, Assumptions, and Mitigations

Risks

- Inconsistent CustId formats across source files causing join failures.
- Unexpected data volume spikes impacting performance.
- Ambiguity in SCD behavior vis-a-vis order-date association causing misattribution.

Assumptions

- Source CSVs conform roughly to provided schemas.
- PricePerUnit and Qty are convertible to numeric types.
- The business expects SCD Type 2 semantics for customer attribute history.

Mitigations

- Implement data validation and normalization for CustId formats.
- Implement throttling, autoscaling and test at scale.
- Confirm SCD semantics (association of orders to customer version) with business stakeholders and document.

17. Implementation Plan and Timeline (High-level)

Phase 1 — Design & Validation (1–2 weeks)

- Confirm business rules (SCD semantics, retention, PII handling).
- Finalize schema and DLT pipeline design.

Phase 2 — Development (2–3 weeks)

- Implement DLT pipeline: ingestion, cleansing, TotalAmount calc, SCD logic, aggregation.

- Implement audit logging and data quality rules.

Phase 3 — Testing & Validation (1–2 weeks)

- Unit tests, integration tests, reconciliation checks with sample data.

Phase 4 — Backfill & Cutover (1 week)

- Backfill historical data and validate.

Phase 5 — Production Run & Monitoring (ongoing)

- Monitor DLT runs, address issues, optimize performance.

18. Deliverables

- Delta Live Tables pipeline code and deployment artifacts.
- DDL scripts for ordersummary and customeraggregatespend (CREATE TABLE IF NOT EXISTS).
- Data quality rules and monitoring dashboards/alerts.
- Test scripts and reconciliation queries.
- Runbooks for operations and incident response.

19. Open Questions / Decisions Needed

- Confirm exact data types and precision for PricePerUnit, TotalAmount, and Qty.
- Confirm how Date is represented and whether timezones matter.
- Clarify SCD-to-order association rule: Should orders always be associated with the customer version valid at order Date, or should customer changes immediately be applied to all orders?
- Confirm retention policy for SCD history and for Delta VACUUM retention.
- Decide whether to include a surrogate key in ordersummary for internal use.
- Confirm required SLAs (end-to-end processing windows) and expected data volumes.

20. Appendix

A. Original high-level steps (for traceability)

- Step 1: Read source CSV data and load into Delta tables [paths provided]
- Step 2: Source schemas for customer and order (fields enumerated)
- Step 3: Add TotalAmount = PricePerUnit * Qty to order table
- Step 4: Remove NULL/"Null" values and duplicate records in both tables
- Step 5: Create ordersummary table if not exists (catalog/schema/table provided)
- Step 6: Join customer and order on CustId and load into SCD Type 2 ordersummary
- Step 7: ordersummary schema enumerated (fields)

- Step 8: Implement SCD Type 2 behavior for customer changes (mark old inactive, insert new with StartDate/EndDate)
- Step 9: Create customeraggregatespend table if not exists (catalog/schema/table and columns)
- Step 10: Aggregate TotalAmount from ordersummary grouped by Name and Date and load into customeraggregatespend
- Step 11: Implement steps 1–10 using Delta Live Tables

B. Suggested example DLT expectations and quality checks (to implement)

- Expectation: no NULLs in primary key columns (CustId, OrderId)
- Expectation: PricePerUnit >= 0 and Qty >= 0
- Expectation: Date parsable to date type
- Expectation: Row counts > 0 for production runs (alert on zero expected)

C. Contact / Approval

- Document owner: Data Engineering Lead / Senior Business Analyst
- Approval required: Business Sponsor, Data Governance, Data Architect

End of Document

If you want, I can:

- Produce example Delta Live Tables pipeline pseudocode/notebook showing each step.
- Draft SQL DDL and DML templates (CREATE TABLE IF NOT EXISTS, MERGE statements for SCD Type 2, and aggregation SQL).
- Provide recommended test datasets and test scripts for acceptance testing.

Data Flow Diagram (DFD)

